

# 오픈소스 최종보고서

강의실 콘센트 위치 및 상태 안내 프로그램

유동건 · 이규정 · 김범준 · 김덕훈

## 1. 프로젝트 개요

본 보고서는 강의실 콘센트 위치 및 상태 안내 프로그램을 사용자 관점과 관리자 관점에서 종합하여 정리한 최종보고서이다. 기존의 두 보고서가 각각 사용 흐름과 관리 흐름을 설명했다면, 본 문서는 서비스 목적, 주요 기능, 이용 절차, 신고 처리 방식, 기대 효과를 하나의 흐름으로 재구성하였다.

| 구분     | 내용                                                            |
|--------|---------------------------------------------------------------|
| 프로젝트명  | 강의실 콘센트 위치 및 상태 안내 프로그램                                       |
| 목적     | 사용자가 강의실 내 콘센트 위치와 사용 가능 여부를 빠르게 확인하고, 고장 상태를 신고할 수 있도록 지원한다. |
| 사용자 기능 | 건물 및 강의실 선택, 콘센트 위치 확인, 상태 조회, 고장 신고, 정상 복구 재점검 요청            |
| 관리자 기능 | 신고 목록 확인, 신고 콘센트의 정상 처리 또는 고장 확정, 처리 결과 반영                    |
| 핵심 가치  | 시설 이용 편의성 향상, 고장 정보의 실시간 공유, 관리자 처리 절차의 단순화                   |

## 2. 사용자 관점 기능

사용자는 프로그램에 접속한 뒤 건물을 선택하고, 해당 건물의 강의실 목록에서 원하는 강의실을 선택한다. 이후 강의실 도면에서 콘센트 위치를 확인할 수 있으며, 각 콘센트의 상태는 색상과 상태 문구로 구분된다.

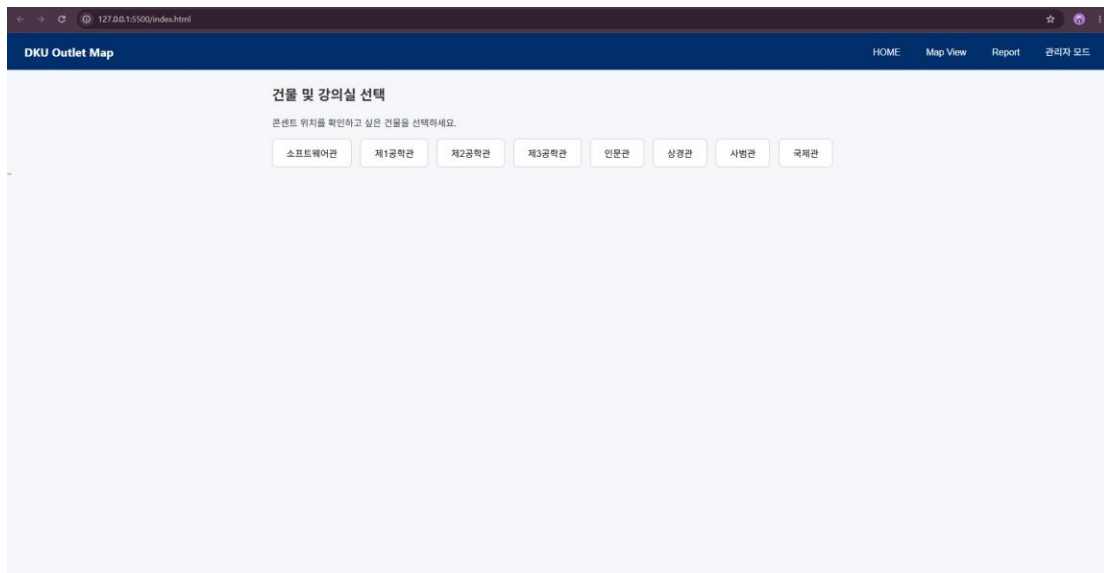


그림 1. 프로그램 접속 후 건물 선택 화면

### 소프트웨어관 516호 콘센트 도면

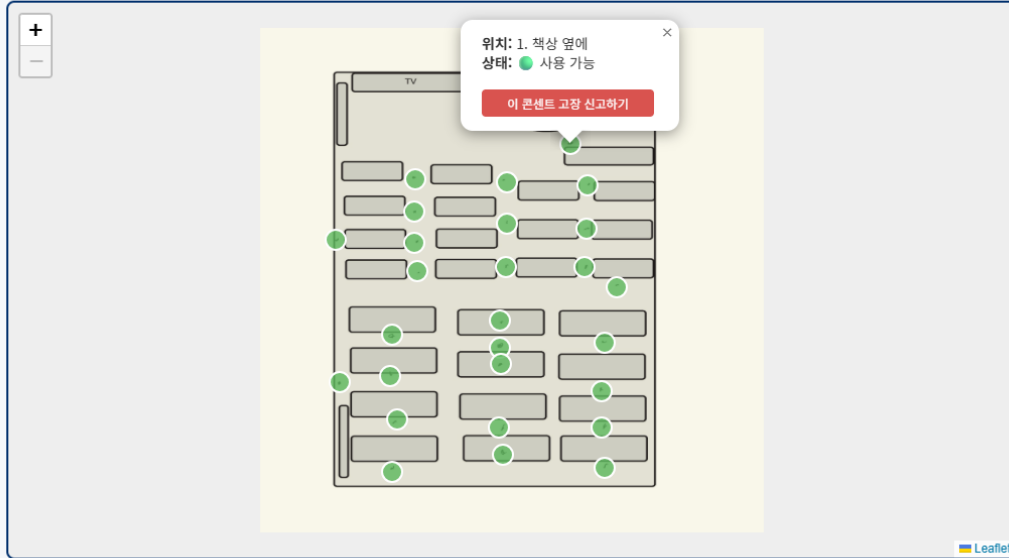


그림 4. 강의실 콘센트 위치 도면

## 고장난 콘센트 신고하기

건물명

소프트웨어관

강의실 호수

516호

콘센트 ID/위치 설명

1. 책상 옆에

신고 접수

그림 5. 콘센트 상세 상태 정보

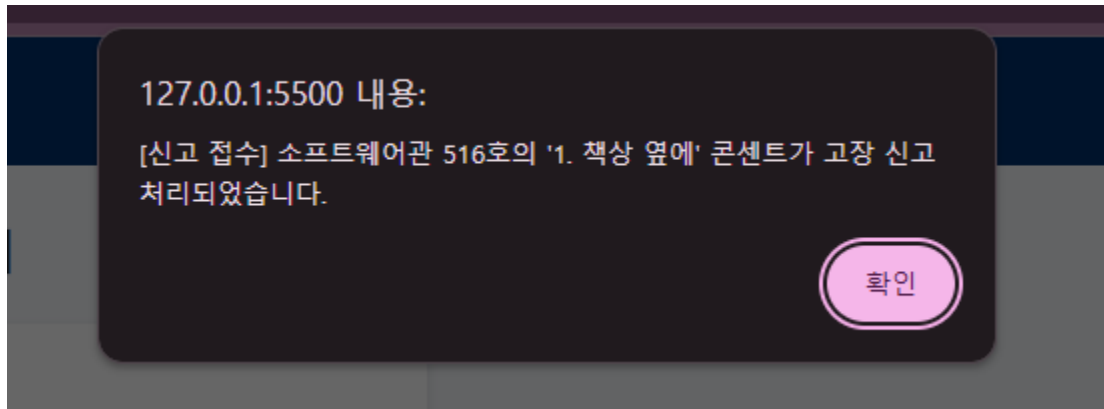


그림 6. 고장 신고 접수 화면

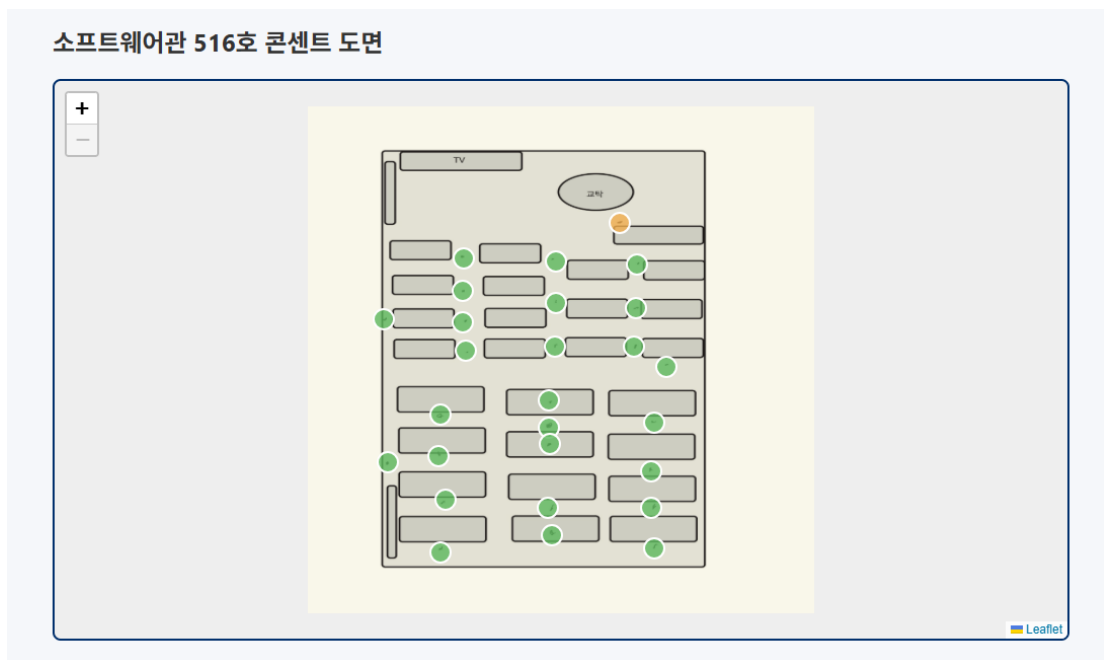


그림 7. 신고 접수 완료 알림

### 소프트웨어관 516호 콘센트 도면

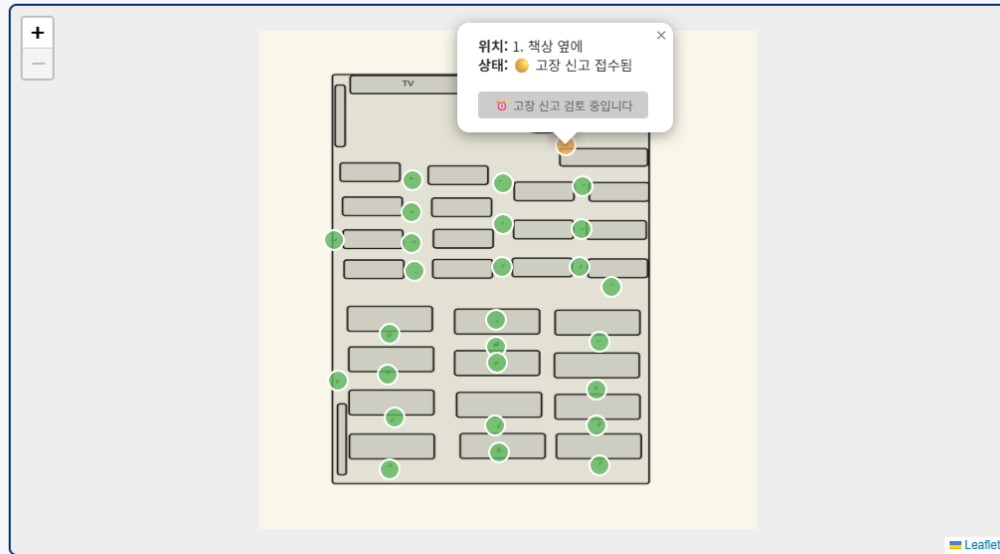


그림 8. 신고 후 검토중 상태로 변경된 콘센트

### 소프트웨어관 516호 콘센트 도면

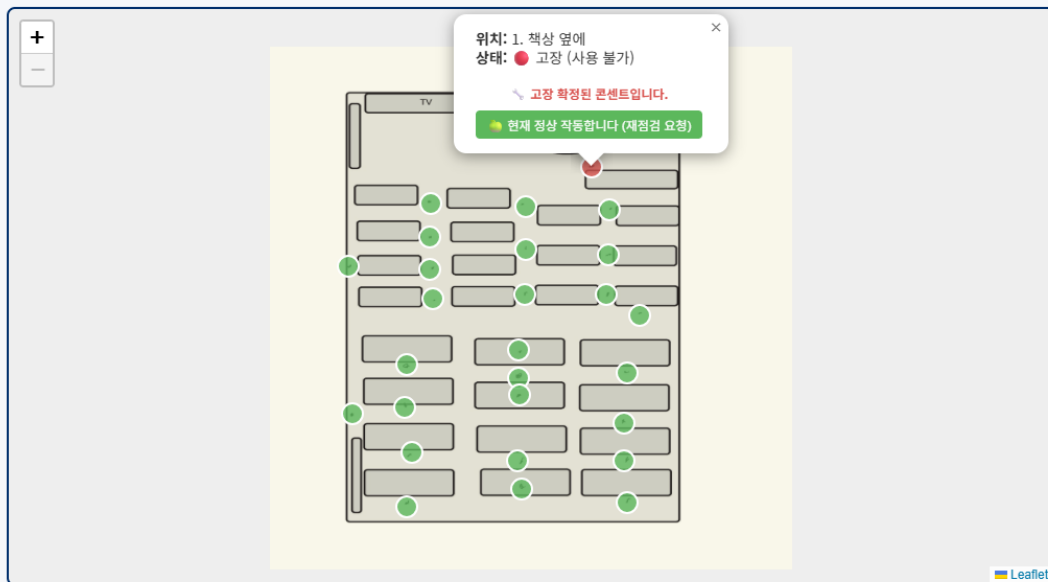


그림 11. 고장 확정 시 사용 불가 상태 표시

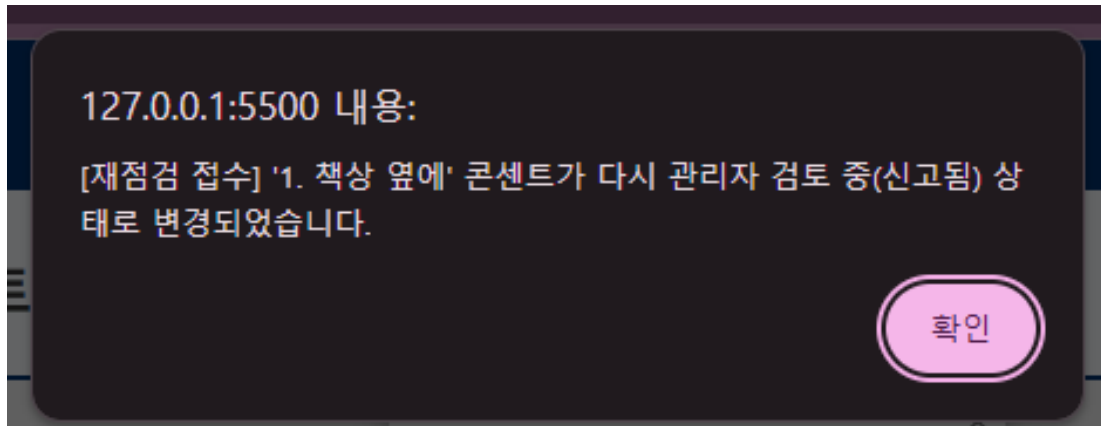


그림 13. 정상 작동 재점검 요청 버튼

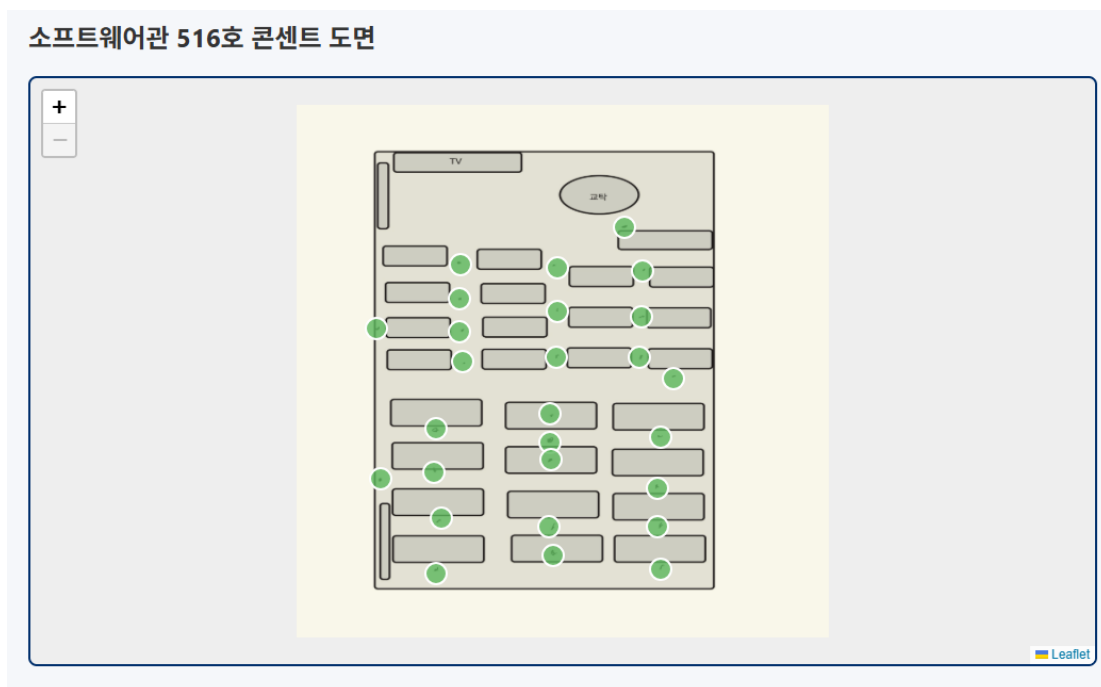


그림 15. 재점검 접수 후 상태 표시

## 2.1 사용자 이용 절차

1. 서버에 접속한 뒤 원하는 건물을 선택한다.
2. 건물 안의 강의실 목록에서 확인하려는 강의실을 선택한다.
3. 강의실 도면에서 초록색으로 표시된 콘센트 위치를 확인한다.
4. 특정 콘센트를 선택해 위치 설명과 현재 상태를 확인한다.
5. 전류가 흐르지 않거나 콘센트가 없는 경우 신고 버튼을 눌러 관리자에게 접수한다.
6. 신고 후에는 콘센트가 검토중 상태로 표시되어 다른 사용자도 현재 상황을 알 수 있다.

### 3. 관리자 관점 기능

관리자는 관리자 모드로 접속한 뒤 비밀번호를 입력하여 로그인한다. 로그인 후에는 사용자가 접수한 고장 신고 목록을 확인할 수 있으며, 각 신고 항목에 대해 정상 처리 또는 고장 확정 중 하나의 조치를 선택한다. 처리 결과는 사용자 화면의 콘센트 상태에 반영된다.

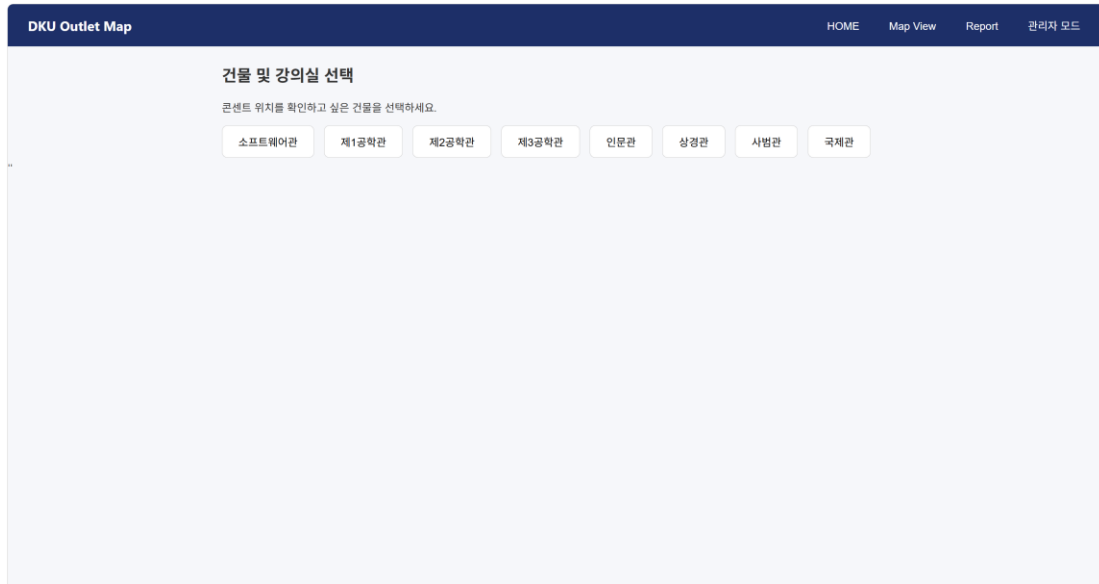


그림 16. 관리자 모드 진입 화면

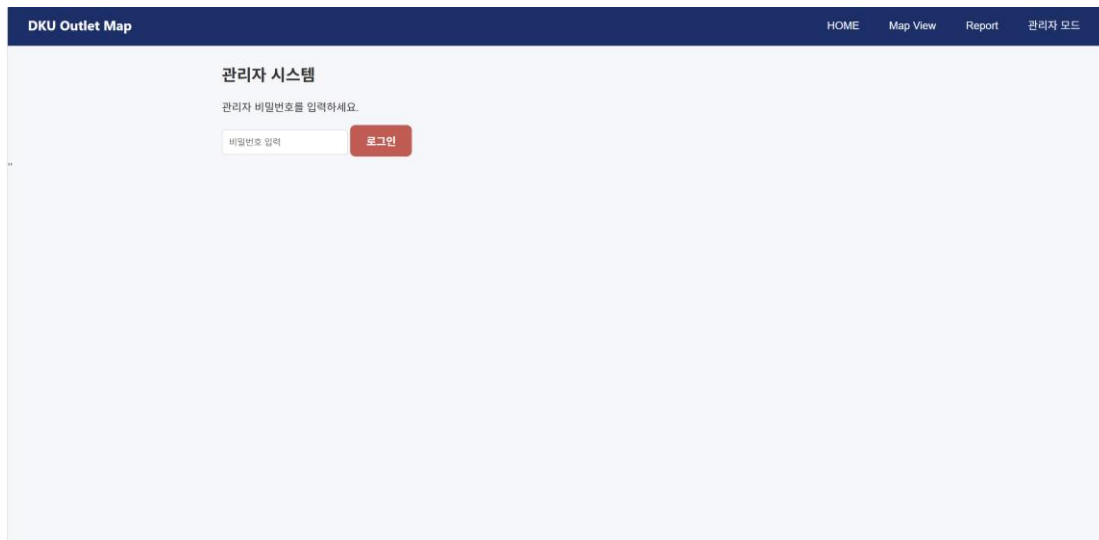


그림 17. 관리자 로그인 화면

| DKU Outlet Map            |           |              |                                         |
|---------------------------|-----------|--------------|-----------------------------------------|
|                           |           |              |                                         |
| 관리자 시스템                   |           |              |                                         |
| 접수된 고장 신고 리스트 (Status: 2) |           |              |                                         |
| 건물                        | 강의실       | 콘센트 위치       | 관리 액션                                   |
| 소프트웨어관                    | 204호(실습실) | 강의 탁자 내부 멀티탭 | <div>정상(0) 처리</div> <div>고장(1) 확장</div> |
| 인문관                       | 301호      | 중앙 통로 바닥     | <div>정상(0) 처리</div> <div>고장(1) 확장</div> |

그림 18. 고장 신고 리스트 화면

127.0.0.1:5500의 메시지

해당 콘센트가 [고장(사용불가)] 상태로 변경되었습니다.

확인

그림 19. 신고 항목 관리 액션 선택



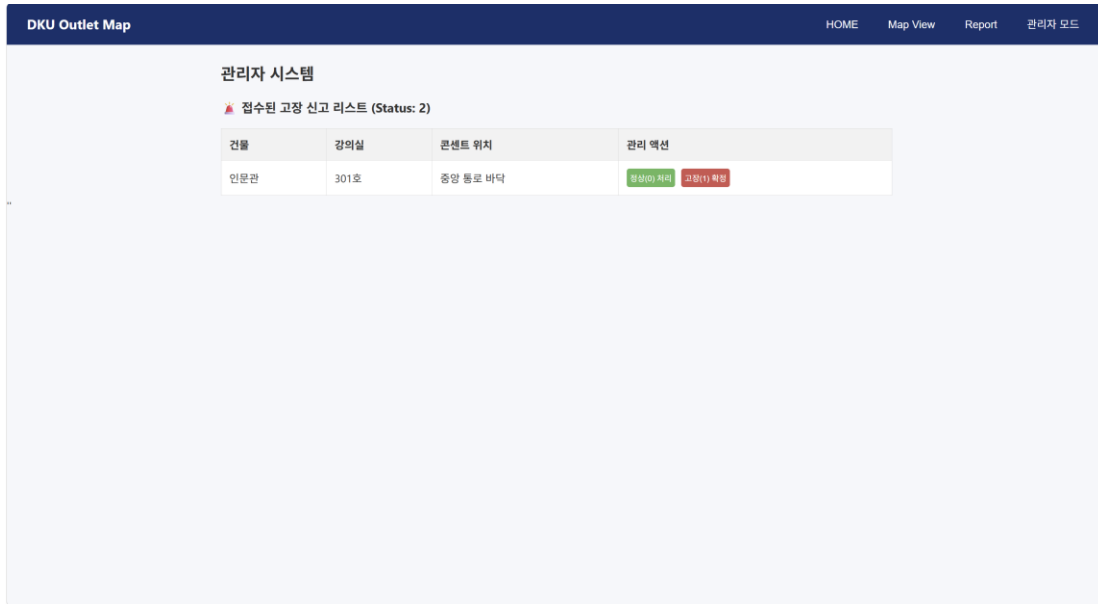


그림 20. 고장 확정 처리 알림

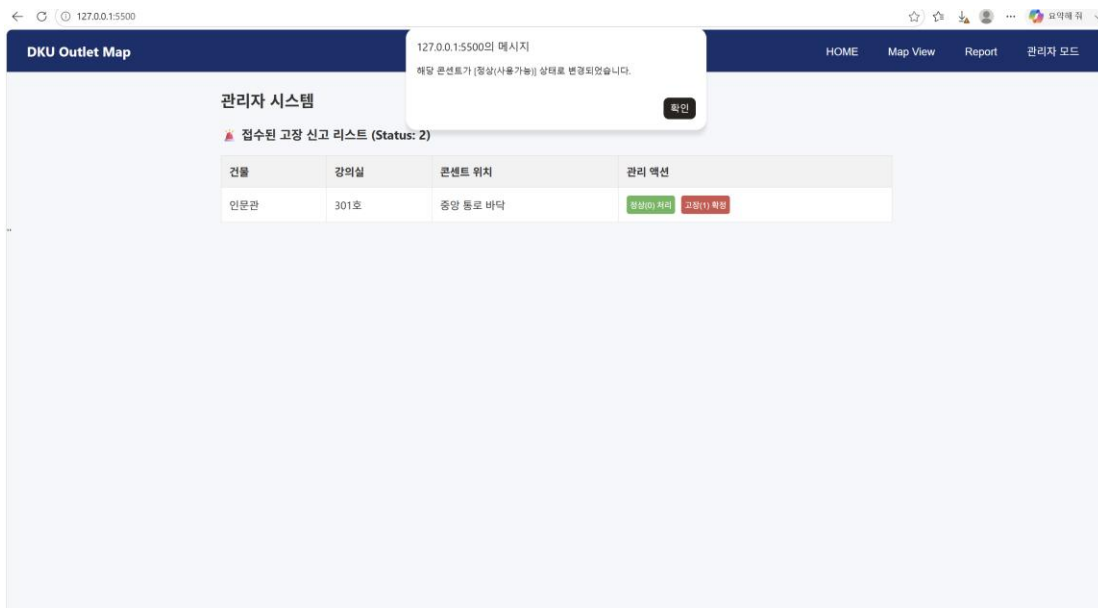


그림 21. 고장 확정 후 리스트 갱신

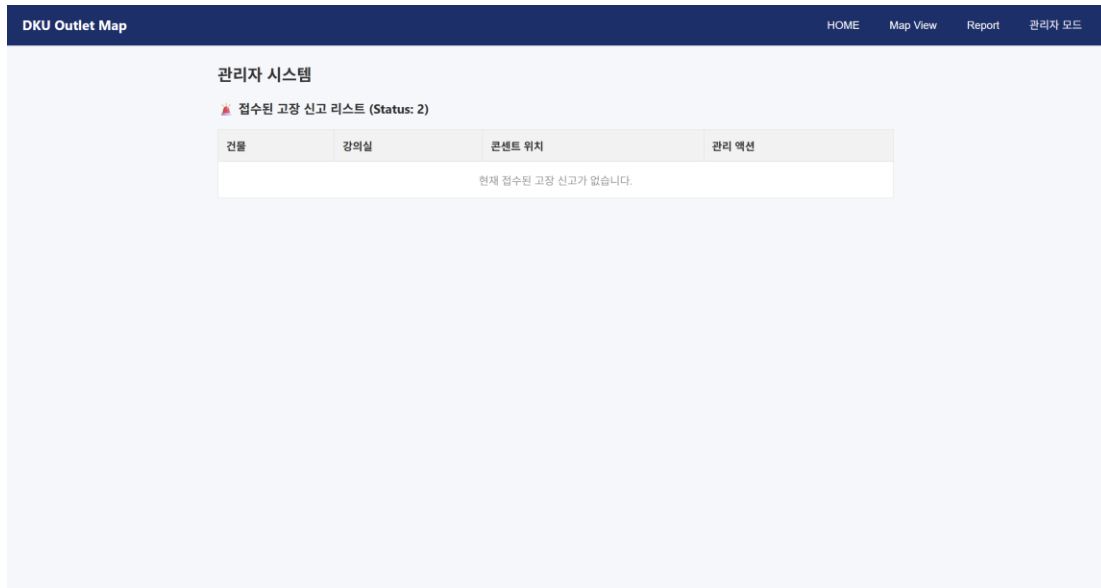


그림 22. 정상 처리 후 신고 목록 정리

### 3.1 관리자 처리 절차

7. 관리자 모드에서 비밀번호를 입력해 관리 화면에 접속한다.
8. 접수된 고장 신고 리스트에서 건물과 신고 항목을 확인한다.
9. 실제 점검 결과에 따라 정상 처리 또는 고장 확정을 선택한다.
10. 정상 처리를 선택하면 해당 콘센트는 사용 가능한 상태로 돌아간다.
11. 고장 확정을 선택하면 해당 콘센트는 고장 및 사용 불가 상태로 표시된다.
12. 처리가 끝난 항목은 신고 리스트에서 제거되어 남은 신고를 쉽게 확인할 수 있다.

## 4. 상태 흐름 및 표시 방식

이 프로그램의 핵심은 콘센트 상태를 사용자와 관리자가 같은 기준으로 확인할 수 있도록 만든 점이다. 색상과 상태 문구를 함께 사용하여 단순 위치 안내를 넘어 시설 관리 흐름까지 연결한다.

| 상태     | 표시 의미       | 후속 동작                                     |
|--------|-------------|-------------------------------------------|
| 정상     | 사용 가능       | 사용자는 콘센트를 바로 이용할 수 있다.                    |
| 검토중    | 신고 접수       | 관리자가 현장 확인 후 정상 처리 또는 고장 확정을 선택한다.        |
| 고장     | 사용 불가       | 사용자는 해당 콘센트 이용을 피하고, 관리자는 수리 및 재점검을 진행한다. |
| 재점검 접수 | 정상 작동 확인 요청 | 사용자가 수리 후 정상 작동을 확인하면 관리자에게 다시 알릴 수 있다.   |

## 5. 종합 평가

- 사용자 입장에서는 강의실에 들어가기 전 콘센트 위치와 사용 가능 여부를 미리 확인할 수 있어 편의성이 높아진다.
- 관리자 입장에서는 신고 항목을 한 화면에서 확인하고 처리할 수 있어 시설 점검 업무가 단순해진다.
- 신고, 검토중, 고장, 정상 복구의 흐름이 화면 상태로 이어져 사용자와 관리자가 같은 정보를 공유할 수 있다.
- 향후에는 신고 시간, 처리 담당자, 처리 완료 일자 등을 추가하면 관리 이력 기능까지 확장할 수 있다.

## 6. 결론

강의실 콘센트 위치 및 상태 안내 프로그램은 단순히 콘센트의 위치를 보여주는 기능에서 그치지 않고, 사용자의 고장 신고와 관리자의 처리 과정을 연결한다는 점에서 실용성이 있다. 사용자는 필요한 콘센트를 빠르게 찾고 고장 여부를 확인할 수 있으며, 관리자는 접수된 문제를 체계적으로 처리할 수 있다. 따라서 본 프로그램은 강의실 이용 편의성과 시설 관리 효율을 함께 높일 수 있는 오픈소스 프로젝트로 평가할 수 있다.

## 7. 코드 구성 및 구현 설명

본 프로그램은 HTML, CSS, JavaScript 세 파일을 중심으로 구성되어 있다. index.html 은 화면 구조를 만들고, style.css 는 화면 디자인을 담당하며, app.js 는 건물 선택, 지도 표시, 콘센트 상태 변경, 신고 접수, 관리자 처리 로직을 실행한다.

### 7.1 파일별 역할

| 파일명        | 분량    | 주요 역할                                                                      |
|------------|-------|----------------------------------------------------------------------------|
| index.html | 93 줄  | 네비게이션 바, HOME/Map View/Report/Admin 섹션, 신고 폼, 관리자 테이블 등 웹 화면의 기본 구조를 정의한다. |
| style.css  | 82 줄  | 상단 메뉴, 버튼, 지도 영역, 신고 폼, 관리자 테이블의 크기·색상·간격을 지정해 사용자가 보기 쉬운 UI 를 만든다.        |
| app.js     | 365 줄 | 건물/강의실/콘센트 데이터를 저장하고, Leaflet 지도를 띄우며, 신고 접수와 관리자 처리에 따라 콘센트 상태를 바꾼다.      |

### 7.2 index.html 설명

index.html 은 사용자가 보는 화면의 뼈대를 담당한다. 상단에는 HOME, Map View, Report, 관리자 모드로 이동하는 네비게이션 버튼이 있고, 본문에는 건물 선택 화면, 지도 화면, 신고 접수 화면, 관리자 화면이 각각 section 태그로 나뉘어 있다. 마지막에는 Leaflet 라이브러리와 app.js 를 불러와 실제 지도 기능과 버튼 동작이 실행되도록 연결한다.

- HOME 섹션: 건물을 선택하고 해당 건물의 강의실 버튼을 생성하는 시작 화면이다.
- Map View 섹션: Leaflet 지도와 강의실 도면 이미지가 표시되는 영역이다.
- Report 섹션: 콘센트 고장 신고를 접수하는 폼이다.
- Admin 섹션: 관리자 비밀번호 입력 후 접수된 신고 목록을 확인하는 화면이다.

### 7.3 style.css 설명

style.css 는 프로그램의 전체적인 화면 분위기와 사용성을 정리한다. 단국대 느낌의 파란색 네비게이션 바를 사용하고, 버튼에는 hover 효과를 적용해 클릭 가능한 요소임을 쉽게 알 수 있게 했다. 지도 영역은 Leaflet 이 정상적으로 보이도록 높이와 너비를 명확히 지정했으며, 신고 폼과 관리자 테이블은 흰색 배경과 테두리를 사용해 정보가 구분되도록 구성했다.

### 7.4 app.js 설명

app.js 는 프로그램의 핵심 기능을 담당한다. 먼저 campusData 에는 건물별 강의실 목록이 저장되고, outletData 에는 강의실별 콘센트 위치 좌표와 상태값이 저장된다. 상태값은 0 이면 정상, 1 이면 고장, 2 이면 신고 접수 상태로 사용된다. 사용자가 건물과 강의실을 선택하면 loadMapPage 함수가 강의실 도면 이미지를 지도 위에 올리고, 각 콘센트 위치에 색상 마커를 표시한다.

- switchTab: HOME, 지도, 신고, 관리자 화면 중 현재 보여줄 화면을 전환한다.

- **selectBuilding**: 선택한 건물에 맞는 강의실 버튼 목록을 동적으로 만든다.
- **loadMapPage**: 강의실 도면 이미지를 **Leaflet** 지도에 표시하고 콘센트 마커와 팝업을 생성한다.
- **handleReportSubmit**: 사용자가 신고를 접수하면 해당 콘센트 상태를 신고 접수 상태로 바꾸고 관리자 목록을 갱신한다.
- **checkAdminPassword/updateAdminDashboard**: 관리자 인증 후 신고 목록을 테이블에 표시한다.
- **changeOutletStatus/revertToReport**: 관리자가 정상 처리 또는 고장 확정을 선택하면 상태값을 변경하고 화면에 반영한다.

## 7.5 코드 동작 흐름

| 단계 | 동작                                          | 관련 코드                                                               |
|----|---------------------------------------------|---------------------------------------------------------------------|
| 1  | 사용자가 건물을 선택하면 강의실 목록이 표시된다.                 | <code>selectBuilding</code>                                         |
| 2  | 강의실을 선택하면 도면 이미지와 콘센트 마커가 지도에 표시된다.         | <code>loadMapPage</code> , <code>outletData</code>                  |
| 3  | 콘센트 마커를 누르면 위치와 상태가 팝업으로 표시된다.              | <code>Leaflet circleMarker</code> , <code>bindPopup</code>          |
| 4  | 고장 신고를 누르면 신고 폼으로 이동하고, 제출 시 상태가 검토중으로 바뀐다. | <code>goToReport</code> , <code>handleReportSubmit</code>           |
| 5  | 관리자는 신고 목록에서 정상 처리 또는 고장 확정을 선택한다.          | <code>updateAdminDashboard</code> , <code>changeOutletStatus</code> |
| 6  | 처리 결과는 다시 지도 마커 색상과 상태 문구에 반영된다.            | <code>loadMapPage</code> 재호출                                        |

## 7.6 구현 화면 및 상태 이미지

아래 이미지는 실제 프로그램에서 사용되는 강의실 도면과 콘센트 상태 표시 이미지이다. 도면 위에 **JavaScript** 로 콘센트 좌표를 배치하고, 상태값에 따라 초록색, 노란색, 빨간색으로 구분하여 사용자와 관리자가 같은 상태 정보를 확인할 수 있게 했다.





그림 24. 콘센트 상태별 표시 이미지

## 7.7 코드 구현 정리

정리하면, 이 프로그램은 **HTML** 로 화면을 나누고 **CSS** 로 사용하기 편한 **UI** 를 구성한 뒤, **JavaScript** 로 실제 기능을 연결한 구조이다. 특히 **app.js** 에서 콘센트 데이터를 객체 형태로 관리하기 때문에 새로운 건물, 강의실, 콘센트 좌표를 추가하기 쉽다. 또한 신고 접수와 관리자 처리가 같은 상태값을 기준으로 동작하므로 사용자 화면과 관리자 화면이 자연스럽게 연결된다.

## 8. 전체 코드 캡처 및 세부 설명

앞의 코드 구성 설명을 보완하기 위해 실제 프로젝트 파일의 전체 코드를 캡처 이미지로 첨부하였다. 각 이미지는 줄번호를 포함하며, 긴 코드는 여러 장으로 나누어 한 줄도 빠지지 않도록 구성하였다.

- index.html: 93 줄, 화면 구조와 외부 라이브러리 연결 담당
- style.css: 82 줄, 전체 화면 디자인과 지도/폼/테이블 스타일 담당
- app.js: 365 줄, 데이터, 지도, 신고, 관리자 처리 기능 담당

### 8.1 index.html 코드 설명

index.html 은 웹사이트의 기본 구조를 담당한다. 화면을 여러 **section** 으로 나누고, 버튼을 누르면 JavaScript 함수가 실행되도록 연결한다.

- head 영역: 문자 인코딩, 반응형 화면 설정, Leaflet CSS, style.css 를 불러온다.
- navbar 영역: HOME, Map View, Report, 관리자 모드로 이동하는 버튼을 배치한다.
- HOME section: 건물 버튼과 강의실 목록이 표시되는 첫 화면이다.
- Map View section: 강의실 도면과 콘센트 마커가 표시되는 지도 영역이다.
- Report section: 선택한 콘센트의 고장 신고를 접수하는 입력 폼이다.
- Admin section: 관리자 비밀번호 입력과 신고 목록 테이블을 보여준다.
- script 영역: Leaflet 라이브러리와 app.js 를 불러와 실제 동작을 연결한다.



| index.html   Code capture 1/3 |                                                                                   |
|-------------------------------|-----------------------------------------------------------------------------------|
| 1                             | <!DOCTYPE html>                                                                   |
| 2                             | <html lang="ko">                                                                  |
| 3                             | <head>                                                                            |
| 4                             | <meta charset="UTF-8">                                                            |
| 5                             | <meta name="viewport" content="width=device-width, initial-scale=1.0">            |
| 6                             | <title>단국대 강의실 콘서트 지도</title>                                                     |
| 7                             | <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" /> |
| 8                             | <link rel="stylesheet" href="style.css">                                          |
| 9                             | </head>                                                                           |
| 10                            | <body>                                                                            |
| 11                            |                                                                                   |
| 12                            | <nav class="navbar">                                                              |
| 13                            | <div class="logo">DKU Outlet Map</div>                                            |
| 14                            | <div class="nav-links">                                                           |
| 15                            | <button onclick="switchTab('home')">HOME</button>                                 |
| 16                            | <button onclick="switchTab('map-view')">Map View</button>                         |
| 17                            | <button onclick="switchTab('report')">Report</button>                             |
| 18                            | <button onclick="switchTab('admin')">관리자 모드</button>                              |
| 19                            | </div>                                                                            |
| 20                            | </nav>                                                                            |
| 21                            |                                                                                   |
| 22                            | <main class="content">                                                            |
| 23                            |                                                                                   |
| 24                            | <section id="tab-home" class="tab-content active">                                |
| 25                            | <h2>건물 및 강의실 선택</h2>                                                              |
| 26                            | <p>콘서트 위치를 확인하고 싶은 건물을 선택하세요.</p>                                                 |
| 27                            | <div class="building-list">                                                       |
| 28                            | <button onclick="selectBuilding('소프트웨어관')">소프트웨어관</button>                        |
| 29                            | <button onclick="selectBuilding('제1공학관')">제1공학관</button>                          |
| 30                            | <button onclick="selectBuilding('제2공학관')">제2공학관</button>                          |
| 31                            | <button onclick="selectBuilding('제3공학관')">제3공학관</button>                          |
| 32                            | <button onclick="selectBuilding('인문관')">인문관</button>                              |
| 33                            | <button onclick="selectBuilding('상경관')">상경관</button>                              |
| 34                            | <button onclick="selectBuilding('사법관')">사법관</button>                              |
| 35                            | <button onclick="selectBuilding('국제관')">국제관</button>                              |
| 36                            | </div>                                                                            |
| 37                            | <div id="classroom-section" class="classroom-list-container hidden">              |
| 38                            | <h3 id="selected-building-title">강의실 선택</h3>                                      |
| 39                            | <div id="classroom-list" class="classroom-list">                                  |
| 40                            | </div>                                                                            |
| 41                            | </div>                                                                            |
| 42                            | </section>                                                                        |

index.html: all source lines included, split for readability

그림 25-1. index.html 전체 코드 캡처 (1/3)

```
index.html | Code capture 2/3
43
44     <section id="tab-map-view" class="tab-content">
45       <h2 id="map-title">강의실 도면 보기</h2>
46       <div id="map"></div> </section>
47
48     <section id="tab-report" class="tab-content">
49       <h2>고장난 콘센트 신고하기</h2>
50       <form id="report-form" onsubmit="handleReportSubmit(event)">
51         <label>건물명</label>
52         <input type="text" id="report-building" readonly required>
53
54         <label>강의실 호수</label>
55         <input type="text" id="report-classroom" readonly required>
56
57         <label>콘센트 ID/위치 설명</label>
58         <input type="text" id="report-location" placeholder="지도에서 핀을 클릭해 신고하거나 직접 작성하세요." required>
59
60         <button type="submit" class="btn-submit">신고 접수</button>
61       </form>
62     </section>
63
64     <section id="tab-admin" class="tab-content">
65       <h2>관리자 시스템</h2>
66       <div id="admin-auth">
67         <p>관리자 비밀번호를 입력하세요.</p>
68         <input type="password" id="admin-password" placeholder="비밀번호 입력">
69         <button onclick="checkAdminPassword()" class="btn-submit">로그인</button>
70       </div>
71       <div id="admin-dashboard" class="hidden">
72         <h3>□ 접수된 고장 신고 리스트 (Status: 2)</h3>
73         <table class="admin-table">
74           <thead>
75             <tr>
76               <th>건물</th>
77               <th>강의실</th>
78               <th>콘센트 위치</th>
79               <th>관리 액션</th>
80             </tr>
81           </thead>
82           <tbody id="admin-report-list">
83             <tbody>
84             </tbody>
85           </tbody>
86         </table>
87
88       </div>
89     </section>
90
91     </main>
92
93     <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
94     <script src="app.js"></script>
95   </body>
96 </html>
```

index.html: all source lines included, split for readability

그림 25-2. index.html 전체 코드 캡처 (2/3)

```
index.html | Code capture 3/3
85   </div>
86 </section>
87
88 </main>
89
90   <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
91   <script src="app.js"></script>
92 </body>
93 </html>
```

index.html: all source lines included, split for readability

그림 25-3. index.html 전체 코드 캡처 (3/3)

## 8.2 style.css 코드 설명

style.css 는 프로그램 화면을 보기 좋고 사용하기 쉽게 만드는 역할을 한다. 전체 배경, 버튼, 지도, 신고 폼, 관리자 테이블의 디자인을 정리한다.

- **body**: 기본 글꼴, 배경색, 글자색을 지정해 전체 분위기를 통일한다.
- **navbar**: 상단 메뉴를 파란색 바 형태로 만들고 버튼을 가로로 배치한다.

- **tab-content** 와 **hidden**: 현재 선택된 화면만 보이도록 **display** 값을 제어한다.
- **button**: 건물과 강의실 선택 버튼에 크기, 테두리, **hover** 효과를 적용한다.
- **#map**: Leaflet 지도가 정상 표시되도록 지도 영역의 높이와 너비를 고정한다.
- **form** 과 **input**: 신고 접수 화면을 카드 형태로 정리하고 입력칸 간격을 맞춘다.
- **admin-table**: 관리자 신고 목록을 표 형태로 보기 쉽게 만든다.

```

style.css | Code capture 1/2
1  /* 기본 세팅 */
2  body {
3      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
4      margin: 0;
5      padding: 0;
6      background-color: #f5f7fa;
7      color: #333;
8  }
9
10 /* 네비게이션 바 */
11 .navbar {
12     display: flex;
13     justify-content: space-between;
14     align-items: center;
15     background-color: #002f6c; /* 단국대 느낌의 블루 */
16     padding: 15px 30px;
17     color: white;
18 }
19 .logo { font-size: 20px; font-weight: bold; }
20 .nav-links button {
21     background: none;
22     border: none;
23     color: white;
24     font-size: 16px;
25     margin-left: 20px;
26     cursor: pointer;
27     padding: 5px 10px;
28 }
29 .nav-links button:hover { background-color: rgba(255,255,255,0.1); border-radius: 4px; }
30
31 /* 메인 콘텐츠 */
32 .content { padding: 4px 40px; max-width: 1000px; margin: 0 auto; }
33 .tab-content { display: none; }
34 .tab-content.active { display: block; }
35
36 /* 버튼 및 리스트 디자인 */
37 .building-list, .classroom-list { display: flex; gap: 15px; margin-top: 15px; flex-wrap: wrap; }
38 button {
39     padding: 12px 24px;
40     font-size: 16px;
41     border: 1px solid #ccc;
42     background-color: white;

```

style.css: all source lines included, split for readability

그림 26-1. style.css 전체 코드 캡처 (1/2)

```
style.css | Code capture 2/2
43     border-radius: 8px;
44     cursor: pointer;
45     transition: all 0.2s;
46 }
47 button:hover { background-color: #002f6c; color: white; border-color: #002f6c; }
48
49 /* 숨김 처리용 클래스 */
50 .hidden { display: none !important; }
51
52 /* 지도 영역 설정 (매우 중요: 크기가 없으면 Leaflet 지도가 안 보임) */
53 #map {
54     height: 550px;
55     width: 100%;
56     background-color: #eee;
57     border: 2px solid #002f6c;
58     border-radius: 8px;
59     margin-top: 20px;
60 }
61
62 /* 폼 스타일 */
63 form { display: flex; flex-direction: column; max-width: 400px; gap: 10px; background: white; padding: 20px; border-radius: 8px; box-shadow: 0 2px 5px rgba(0,0,0,0.1); }
64 input { padding: 10px; border: 1px solid #ccc; border-radius: 4px; }
65 .btn-submit { background-color: #d9534f; color: white; border: none; font-weight: bold; }
66 .btn-submit:hover { background-color: #c9302c; }
67
68 /* 관리자 테이블 */
69 .admin-table { width: 100%; border-collapse: collapse; background: white; margin-top: 15px; }
70 .admin-table th, .admin-table td { border: 1px solid #ddd; padding: 12px; text-align: left; }
71 .admin-table th { background-color: #f2f2f2; }
72
73 /* 지도가 그려지는 영역 안의 이미지가 마우스 클릭을 방해하지 못하게 설정 */
74 .leaflet-image-layer, img {
75     pointer-events: none !important;
76     user-select: none !important; /* 파랗게 선택되는 현상 방지 */
77 }
78
79 /* 대신 지도 자체는 클릭을 받아야 하므로 설정 */
80 .leaflet-container {
81     pointer-events: auto !important;
82 }
83
84 style.css: all source lines included, split for readability
```

그림 26-2. style.css 전체 코드 캡처 (2/2)

## 8.3 app.js 코드 설명

app.js 는 프로젝트의 실제 기능을 담당하는 핵심 파일이다. 데이터 저장, 화면 전환, 지도 생성, 신고 접수, 관리자 처리까지 대부분의 동작이 이 파일에서 실행된다.

- 전역 변수: 현재 지도, 선택한 건물, 선택한 강의실 정보를 저장한다.
- campusData: 건물별 강의실 목록을 저장해 건물 선택 시 강의실 버튼을 만들 수 있게 한다.
- outletData: 각 강의실의 콘센트 ID, 위치 설명, 좌표, 상태값을 저장한다.
- switchTab: 사용자가 누른 메뉴에 따라 HOME, 지도, 신고, 관리자 화면을 전환한다.
- selectBuilding: 선택한 건물의 강의실 목록을 동적으로 생성한다.
- loadMapPage: 강의실 도면 이미지를 Leaflet 지도에 올리고, 콘센트 좌표에 색상 마커를 표시한다.
- goToReport 와 handleReportSubmit: 신고 화면으로 이동하고, 신고 접수 시 콘센트 상태를 검토중으로 변경한다.

- **checkAdminPassword** 와 **updateAdminDashboard**: 관리자 인증 후 접수된 신고 목록을 관리자 테이블에 출력한다.
- **changeOutletStatus**: 관리자가 정상 처리 또는 고장 확정을 선택하면 상태값을 바꾸고 지도 화면에 다시 반영한다.
- **revertToReport**: 고장 확정된 콘센트가 다시 정상 작동할 때 재점검 요청 상태로 돌리는 기능이다.

```

app.js | Code capture 1/9
1 // =====
2 // 1. 전역 변수 및 샘플 데이터 설정
3 // =====
4 let currentMap = null; // Leaflet 지도 객체를 담을 변수
5 let selectedBuilding = "";
6 let selectedClassroom = "";
7
8 // 단국대 빌딩별 강의실 데이터
9 const campusData = {
10   "소프트웨어관": ["204호(실습실)", "301호", "516호"],
11   "제1공학관": ["101호", "102호"],
12   "제2공학관": ["201호", "202호"],
13   "제3공학관": ["105호"],
14   "인문관": ["301호", "302호"],
15   "상경관": ["201호"],
16   "사범관": ["401호"],
17   "국제관": ["101호(글로벌라운지)"]
18 };
19
20 // 강의실별 콘센트 가상 좌표 및 상태 데이터 (Status -> 0: 정상, 1: 고장, 2: 신고됨)
21 // ※ 대피도 이미지를 1000x1000 격자로 가정했을 때의 좌표 [Y축, X축]
22 const outletData = {
23   "소프트웨어관_204호(실습실)": [
24     { id: "sw204-1", name: "창가 해커톤 테이블 아래", coords: [300, 150], status: 0 },
25     { id: "sw204-2", name: "중앙 분임조 좌석 A", coords: [500, 500], status: 0 },
26     { id: "sw204-3", name: "강의 탁자 내부 멀티탭", coords: [850, 450], status: 2 } // 신고됨
27   ],
28   "소프트웨어관_301호": [
29     { id: "sw301-1", name: "뒤편 사물함 옆", coords: [150, 800], status: 0 }
30   ],
31
32   "제2공학관_201호": [
33     { id: "eng2-201-1", name: "중앙 기동 콘센트", coords: [450, 450], status: 0 },
34     { id: "eng2-201-2", name: "창측 맨 뒷자리", coords: [100, 100], status: 1 } // 고장
35   ],
36
37   "제3공학관_105호": [
38     { id: "eng3-105-1", name: "복도측 벽면 4구", coords: [600, 900], status: 0 }
39   ],
40
41   "사범관_401호": [
42     { id: "edu401-1", name: "교탁 옆 벽면", coords: [800, 200], status: 0 }
43   ]
44 };

```

app.js: all source lines included, split for readability

그림 27-1. app.js 전체 코드 캡처 (1/9)

| app.js   Code capture 2/9 |                                                                            |
|---------------------------|----------------------------------------------------------------------------|
| 43                        | },                                                                         |
| 44                        |                                                                            |
| 45                        | "국제관_101호(글로벌라운지)": [                                                      |
| 46                        | { id: "int101-1", name: "원형 소파 아래 매립형", coords: [500, 500], status: 0 },   |
| 47                        | { id: "int101-2", name: "창가 바 테이블 오른쪽", coords: [300, 950], status: 0 }    |
| 48                        | ],                                                                         |
| 49                        | "제1공학관_101호": [ // 캠퍼스 데이터 매칭을 위해 명칭 통일                                    |
| 50                        | { id: "eng101-1", name: "칠판 왼쪽 구석", coords: [850, 150], status: 0 },       |
| 51                        | { id: "eng101-2", name: "뒷문 뒀 기둥", coords: [200, 850], status: 0 },        |
| 52                        | { id: "eng101-3", name: "창가 중간 자리", coords: [500, 50], status: 1 } // 고장   |
| 53                        | ],                                                                         |
| 54                        | "제1공학관_102호": [                                                            |
| 55                        | { id: "eng102-1", name: "교탁 아래", coords: [800, 500], status: 0 }           |
| 56                        | ],                                                                         |
| 57                        | "인문관_301호": [                                                              |
| 58                        | { id: "hum301-1", name: "중앙 통로 바닥", coords: [450, 500], status: 2 } // 신고됨 |
| 59                        | ],                                                                         |
| 60                        | "소프트웨어관_516호" : [                                                          |
| 61                        | { id: "sw516-1", name: "1. 책상 뒀에", coords: [771, 616], status: 0 },        |
| 62                        | { id: "sw516-2", name: "2. 책상 뒀에", coords: [701, 308], status: 0 },        |
| 63                        | { id: "sw516-3", name: "3. 책상 뒀에", coords: [695, 490], status: 0 },        |
| 64                        | { id: "sw516-4", name: "4. 책상 뒀에", coords: [689, 650], status: 0 },        |
| 65                        | { id: "sw516-5", name: "5. 책상 뒀에", coords: [637, 306], status: 0 },        |
| 66                        | { id: "sw516-6", name: "6. 책상 뒀에", coords: [613, 490], status: 0 },        |
| 67                        | { id: "sw516-7", name: "7. 벽면", coords: [603, 648], status: 0 },           |
| 68                        | { id: "sw516-8", name: "8. 책상 뒀에", coords: [581, 150], status: 0 },        |
| 69                        | { id: "sw516-9", name: "9. 책상 뒀에", coords: [575, 306], status: 0 },        |
| 70                        | { id: "sw516-10", name: "10. 책상 뒀에", coords: [519, 312], status: 0 },      |
| 71                        | { id: "sw516-11", name: "11. 책상 뒀에", coords: [527, 488], status: 0 },      |
| 72                        | { id: "sw516-12", name: "12. 책상 뒀에", coords: [527, 644], status: 0 },      |
| 73                        | { id: "sw516-13", name: "13. 책상 뒀에", coords: [487, 707], status: 0 },      |
| 74                        | { id: "sw516-14", name: "14. 책상 뒀에", coords: [393, 262], status: 0 },      |
| 75                        | { id: "sw516-15", name: "15. 책상 위예", coords: [421, 476], status: 0 },      |
| 76                        | { id: "sw516-16", name: "16. 책상 뒀에", coords: [367, 476], status: 0 },      |
| 77                        | { id: "sw516-17", name: "17. 책상 뒀에", coords: [377, 683], status: 0 },      |
| 78                        | { id: "sw516-18", name: "18. 책상 뒀에", coords: [299, 158], status: 0 },      |
| 79                        | { id: "sw516-19", name: "19. 책상 뒀에", coords: [311, 258], status: 0 },      |
| 80                        | { id: "sw516-20", name: "20. 책상 위예", coords: [335, 478], status: 0 },      |
| 81                        | { id: "sw516-21", name: "21. 책상 뒀에", coords: [281, 677], status: 0 },      |
| 82                        | { id: "sw516-22", name: "22. 책상 뒀에", coords: [225, 272], status: 0 },      |
| 83                        | { id: "sw516-23", name: "23. 책상 뒀에", coords: [209, 474], status: 0 },      |
| 84                        | { id: "sw516-24", name: "24. 책상 뒀에", coords: [209, 677], status: 0 },      |

app.js: all source lines included, split for readability

그림 27-2. app.js 전체 코드 캡처 (2/9)

app.js | Code capture 3/9

```
85         { id: "sw516-25", name: "25. 책상 옆에", coords: [121, 262], status: 0 },
86         { id: "sw516-26", name: "26. 책상 위에", coords: [155, 482], status: 0 },
87         { id: "sw516-27", name: "27. 책상 옆에", coords: [129, 683], status: 0 }
88     ]
89 };
90
91 // =====
92 // 2 탭 전환 및 메뉴 제어 로직
93 // =====
94 function switchTab(tabId) {
95     document.querySelectorAll('tab-content').forEach(tab => {
96         tab.classList.remove('active');
97     });
98     document.getElementById('tab-' + tabId).classList.add('active');
99
100     if (tabId === 'map-view' && currentMap) {
101         setTimeout(() => { currentMap.invalidateSize(); }, 100);
102     }
103 }
104
105 // HOME에서 건물 선택 시 강의실 리스트 등장
106 function selectBuilding(buildingName) {
107     selectedBuilding = buildingName;
108     document.getElementById('selected-building-title').innerText = `${buildingName} - 강의실 선택`;
109
110     const classroomListDiv = document.getElementById('classroom-list');
111     classroomListDiv.innerHTML = "";
112
113     if (campusData[buildingName]) {
114         campusData[buildingName].forEach(room => {
115             const btn = document.createElement('button');
116             btn.innerText = room;
117             btn.onclick = () => loadMapPage(buildingName, room);
118             classroomListDiv.appendChild(btn);
119         });
120     }
121
122     document.getElementById('classroom-section').classList.remove('hidden');
123 }
124
125 // =====
126 // 3. Leaflet.js 실내 지도 구현 (핵심)
```

app.js: all source lines included, split for readability

그림 27-3. app.js 전체 코드 캡처 (3/9)

app.js | Code capture 4/9

```
127 // =====
128 function loadMapPage(building, room) {
129     selectedBuilding = building; // 상태 업데이트를 위한 동기화
130     selectedClassroom = room;
131     switchTab('map-view');
132     document.getElementById('map-title').innerText = `${building} ${room} 콘센트 도면`;
133
134     if (currentMap) {
135         currentMap.remove();
136         currentMap = null;
137     }
138
139     currentMap = L.map('map', {
140         crs: L.CRS.Simple,
141         minZoom: -1,
142         maxZoom: 2
143     });
144
145     const bounds = [[0, 0], [1000, 1000]];
146     const imagePath = `images/${building}_${room}.jpg`;
147
148     L.imageOverlay(imagePath, bounds).addTo(currentMap);
149     currentMap.fitBounds(bounds);
150
151     const dataKey = `${building}_${room}`;
152     if (outletData[dataKey]) {
153         outletData[dataKey].forEach(outlet => {
154             let markerColor = "#5cb85c"; // 정상 (Green)
155             let statusText = "□ 사용 가능";
156             let buttonHtml = ""; // □ 블록 스코프 안전 선언
157
158             if (outlet.status === 1) {
159                 markerColor = "#d9534f"; // 고장 (Red)
160                 statusText = "□ 고장 (사용 불가)";
161
162                 // □ [변경] 고장 확정 상태일 때 사용자가 다시 정상 작동 상태로 재요청할 수 있는 버튼 제공
163                 buttonHtml = `
164                     <div style="text-align:center, color:#d9534f, font-weight:bold, margin-bottom:8px, font-size:12px;">
165                         □ 고장 확정된 콘센트입니다.
166                     </div>
167                     <button onclick="revertToReport('${building}', '${room}', '${outlet.id}')"
168                         style="padding:5px 10px; font-size:12px; background-color:#5cb85c; color:white; border:none; border-radius:4px; cursor:pointer; width:10
```

app.js: all source lines included, split for readability

그림 27-4. app.js 전체 코드 캡처 (4/9)



```

0%,">
169         □ 현재 정상 작동합니다 (재점검 요청)
170     </button>
171     `;
172     } else if (outlet.status === 2) {
173         markerColor = "#f0ad4e"; // 신고됨 (Yellow)
174         statusText = "□ 고장 신고 접수됨";
175
176         // □ 대기 상태 시 중복 신고를 막기 위해 비활성화(disabled) 처리
177         buttonHtml = `
178             <button disabled
179                 style="padding:5px 10px; font-size:12px; background-color:#ccc; color:#666; border:none; border-radius:4px; width:100%; cursor:not-allow
ed;">
180                 □ 고장 신고 검토 중입니다
181             </button>
182             `;
183     } else {
184         // □ 정상 상태일 때만 신고하기 버튼 활성화
185         buttonHtml = `
186             <button onclick="goToReport('${building}', '${room}', '${outlet.name}')">
187                 □ 고장 신고하기
188                 style="padding:5px 10px; font-size:12px; background-color:#d9534f; color:white; border:none; border-radius:4px; cursor:pointer; width:100%;
0%,">
188         이 콘센트 고장 신고하기
189     </button>
190     `;
191     }
192
193     // 마커 생성 및 지도 추가
194     const marker = L.circleMarker(outlet.coords, {
195         radius: 10,
196         fillColor: markerColor,
197         color: "#fff",
198         weight: 2,
199         fillOpacity: 0.8
200     }).addTo(currentMap);
201
202     // □□ [버그 수정 완료] 하드코딩 버튼을 제거하고 동적으로 가공된 ${buttonHtml} 삽입
203     const popupContent = `
204         <div style="font-size:14px; font-family: sans-serif; min-width: 170px;">
205             <b>위치:</b> ${outlet.name}<br>
206             <b>상태:</b> ${statusText}<br><br>
207             ${buttonHtml}

```

app.js: all source lines included, split for readability

그림 27-5. app.js 전체 코드 캡처 (5/9)

```

208     </div>
209     `;
210     marker.bindPopup(popupContent);
211   });
212   }
213 }
214
215 function goToReport(building, room, locationName) {
216   document.getElementById('report-building').value = building;
217   document.getElementById('report-classroom').value = room;
218   document.getElementById('report-location').value = locationName;
219   switchTab('report');
220 }
221
222 // =====
223 // 4. Report (고장 신고) 처리 로직
224 // =====
225 function handleReportSubmit(event) {
226   event.preventDefault();
227
228   const building = document.getElementById('report-building').value;
229   const classroom = document.getElementById('report-classroom').value;
230   const location = document.getElementById('report-location').value;
231
232   const dataKey = `${building}_${classroom}`;
233
234   if (outletData[dataKey]) {
235     const outlet = outletData[dataKey].find(o => o.name === location);
236
237     if (outlet) {
238       outlet.status = 2;
239       alert(`[신고 접수] ${building} ${classroom}의 '${location}' 콘센트가 고장 신고 처리되었습니다.`);
240
241       loadMapPage(building, classroom);
242       updateAdminDashboard();
243       switchTab('map-view');
244     } else {
245       alert("선택하신 위치의 콘센트 정보가 존재하지 않습니다. 지도에서 핀을 먼저 클릭해주세요.");
246     }
247   }
248 }
249

```

app.js: all source lines included, split for readability

그림 27-6. app.js 전체 코드 캡처 (6/9)

app.js | Code capture 7/9

```
250 // =====
251 // 5. 관리자 모드 (인증 및 대시보드 상태 변경)
252 // =====
253 const ADMIN_PASSWORD = "dku1234";
254
255 function checkAdminPassword() {
256   const passwordInput = document.getElementById('admin-password').value;
257
258   if (passwordInput === ADMIN_PASSWORD) {
259     document.getElementById('admin-auth').classList.add('hidden');
260     document.getElementById('admin-dashboard').classList.remove('hidden');
261     updateAdminDashboard();
262   } else {
263     alert("비밀번호가 틀렸습니다. 다시 시도하세요.");
264   }
265 }
266
267 // 관리자 대시보드에 신고된 콘센트(2)와 고장 확정된 콘센트(1) 리스트를 모두 띄우는 함수
268 function updateAdminDashboard() {
269   const tbody = document.getElementById('admin-report-list');
270   if(!tbody) return;
271   tbody.innerHTML = "";
272
273   let hasReports = false;
274
275   for (const key in outletData) {
276     const [building, classroom] = key.split("_");
277
278     outletData[key].forEach(outlet => {
279       if (outlet.status === 2) {
280         hasReports = true;
281         const tr = document.createElement('tr');
282
283         tr.innerHTML = `
284           <td> ${building} </td>
285           <td> ${classroom} </td>
286           <td> ${outlet.name} </td>
287           <td>
288             <button onclick="changeOutletStatus('${building}', '${classroom}', '${outlet.id}', 0)"
289               style="background-color:#5cb85c; color:white; padding:5px; font-size:12px; border:none; border-radius:4px; cursor:pointer;">
290               정상(0) 처리
291             </button>

```

app.js: all source lines included, split for readability

그림 27-7. app.js 전체 코드 캡처 (7/9)

```

292         <button onclick="changeOutletStatus('${building}', '${classroom}', '${outlet.id}', 1)"
293             style="background-color:#d9534f; color:white; padding:5px; font-size:12px; margin-left:5px; border:none; border-radius:4px; cursor:poi
nter,">
294             고장(1) 확정
295         </button>
296     </td>
297     `;
298     tbody.appendChild(tr);
299 }
300 });
301 }
302
303 if (!hasReports) {
304     tbody.innerHTML = `<tr><td colspan="4" style="text-align:center; color:#999; padding:15px;">현재 접수된 고장 신고가 없습니다.</td></tr>`;
305 }
306 }
307
308 function changeOutletStatus(building, classroom, outletId, newStatus) {
309     const dataKey = `${building}_${classroom}`;
310     const outlet = outletData[dataKey].find(o => o.id === outletId);
311
312     if (outlet) {
313         outlet.status = newStatus;
314         const statusText = newStatus === 0 ? "정상(사용가능)" : "고장(사용불가)";
315         alert(`해당 콘센트가 [${statusText}] 상태로 변경되었습니다.`);
316
317         updateAdminDashboard();
318         if (currentMap && selectedBuilding === building && selectedClassroom === classroom) {
319             loadMapPage(building, classroom);
320         }
321     }
322 }
323
324 function revertToReport(building, room, outletId) {
325     const dataKey = `${building}_${room}`;
326
327     if (outletData[dataKey]) {
328         // 이름을 구별하셨든 ID를 쓰시든 안전하게 id 기반으로 찾습니다.
329         // (만약 아직 ID 기반 수정을 안 하셨고 이름만 바꾸셨다면 outletId 자리에 outlet.name이 들어와도 잘 작동합니다)
330         const outlet = outletData[dataKey].find(o => o.id === outletId || o.name === outletId);
331
332         if (outlet) {

```

app.js: all source lines included, split for readability

그림 27-8. app.js 전체 코드 캡처 (8/9)

```
app.js | Code capture 9/9
333 // □ 상태를 다시 2(신고 접수/검토 중)로 변경합니다.
334 outlet.status = 2;
335
336 alert('재점검 접수' '${outlet.name}' 콘센트가 다시 관리자 검토 중(신고됨) 상태로 변경되었습니다.);
337
338 // 화면 즉시 갱신
339 loadMapPage(building, classroom = room);
340 if (typeof updateAdminDashboard === "function") {
341     updateAdminDashboard(); // 관리자 대시보드도 실시간 동기화
342 }
343 }
344 }
345 }
346
347 // =====
348 // 6. 좌표 추출 디버거 리스너
349 // =====
350 window.addEventListener('click', function(e) {
351     if (!currentMap) return;
352
353     const mapCont = document.getElementById('map');
354     if (mapCont && mapCont.contains(e.target)) {
355         if (e.target.classList.contains('leaflet-interactive') && e.target.tagName === 'path') {
356             return;
357         }
358
359         const point = currentMap.mouseEventToLatLng(e);
360         const y = Math.round(point.lat);
361         const x = Math.round(point.lng);
362
363         console.log('%c□ 찾았다 콘센트 좌표 -> coords: [${y}, ${x}], "color: #002f6c; font-weight: bold; font-size: 14px;');
364     }
365 });
app.js: all source lines included, split for readability
```

그림 27-9. app.js 전체 코드 캡처 (9/9)

## 8.4 코드 구현 요약

전체적으로 이 프로그램은 index.html 에서 화면의 기본 틀을 만들고, style.css 에서 사용자 인터페이스를 정리한 뒤, app.js 에서 실제 동작을 제어하는 구조이다. 특히 app.js 는 콘센트 데이터를 객체로 관리하기 때문에 새로운 강의실이나 콘센트 좌표를 추가하기 쉽고, 상태값만 변경하면 사용자 화면과 관리자 화면이 함께 갱신되는 장점이 있다.